V-SIL at a Glance — General Architecture, Gate G0, and Gate G1 (Ctx 02 Design)

David Ramírez-Betancourt — GCPDS, Universidad Nacional de Colombia • 16 October 2026

1 General Methodology Architecture

V-SIL is one lifecycle in three layers: conceptual validation (zz_Workflow), continuous software development (CI+SIL), and post-development operations (PCG). Each layer's terminal output is the next layer's input. Gate G0 (green, deterministic) validates the Ground Truth documents before any human or AI gate runs; only acceptance testing stays at HIL on real hardware.

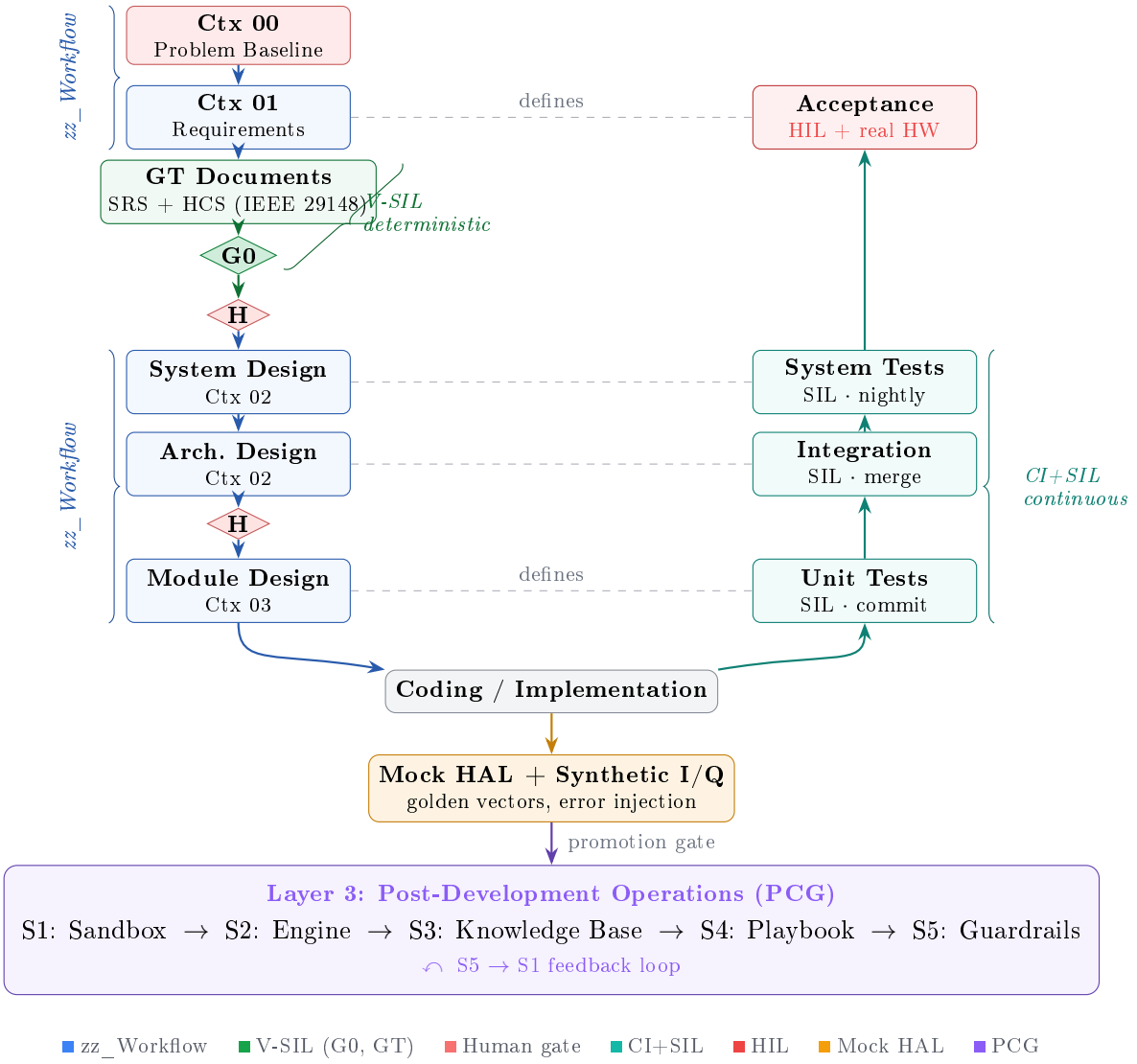


Figure 1: **V-SIL lifecycle.** Blue: conceptual methodology (zz_Workflow); green: deterministic V-SIL additions (GT, Gate G0); coral: human readiness gates; teal: continuous SIL test arm; red: the only HIL level; violet: post-development pipeline.

Table 1: The three layers and their terminal outputs.

#	Name	Source	Purpose	Terminal output
1	Conceptual Validation	zz_Workflow	Define and validate what to build	Verified Prototype + Evidence
2	Software Development	CI+SIL	Implement and continuously verify	Regression-passing build
3	Post-Development Ops	PCG	Govern the deployed system	Audit-ready evidence archive

2 Gate G0 — Structural Validation of Ground Truth

G0 is the first gate and the only fully implemented one: **no AI, pure script** (`g0_validate.py`, stdlib-only, no network, no writes). It answers one question — *does each GT document follow the IEEE 29148 template?* — in four phases, with a binary verdict (exit 0 / exit 1). Errors block; warnings do not. A failure routes back to the GT author (R1) before any human reviewer sees the document.

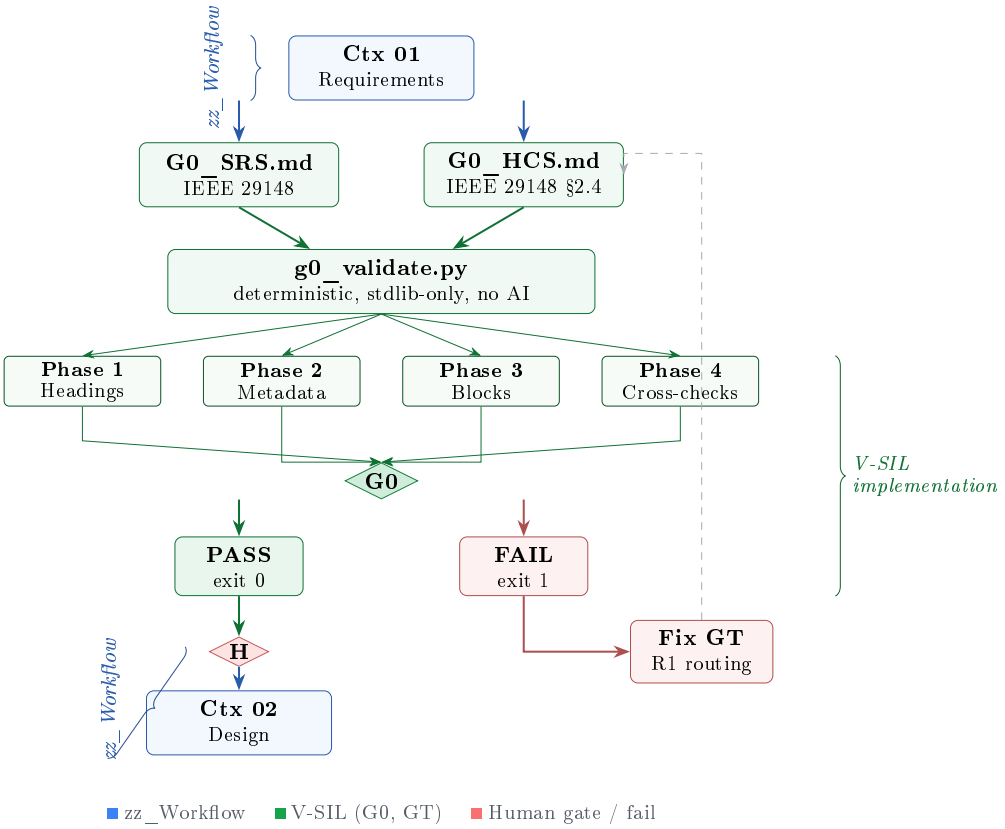


Figure 2: **G0 validation flow.** Ctx 01 produces the GT documents; the script validates them deterministically through four phases. PASS admits them to human Gate H and Ctx 02; FAIL returns to fix the GT structure (R1). No AI is involved.

Table 2: The four G0 validation phases.

Phase	Target	Checks
1	Document headings	All required headings present (substring match on normalized text)
2	Metadata table	Document ID, Project, Organization, SemVer version, Classification — no placeholders
3	Blocks	ID format + uniqueness, required fields, enumerated types, description, ≥ 1 acceptance criterion
4	Cross-checks	SRS: traceability coverage + acyclic dependencies; HCS: no orphan components

3 Gate G1 — Ctx 02 Design Synthesis (GT → Architecture)

`Ctx 00` → `Ctx 01` → `G0` ✓→ `[Ctx 02 • G1]` → `Gate H` → `Ctx 03`

G1 turns the validated GT into a software architecture. No single framework covers the Ctx 02 mandate, so G1 hybrids three: MAAD (multi-hypothesis generation), ARLO (quantitative selection), and ADD (module decomposition) — closing with a human Gate H package.

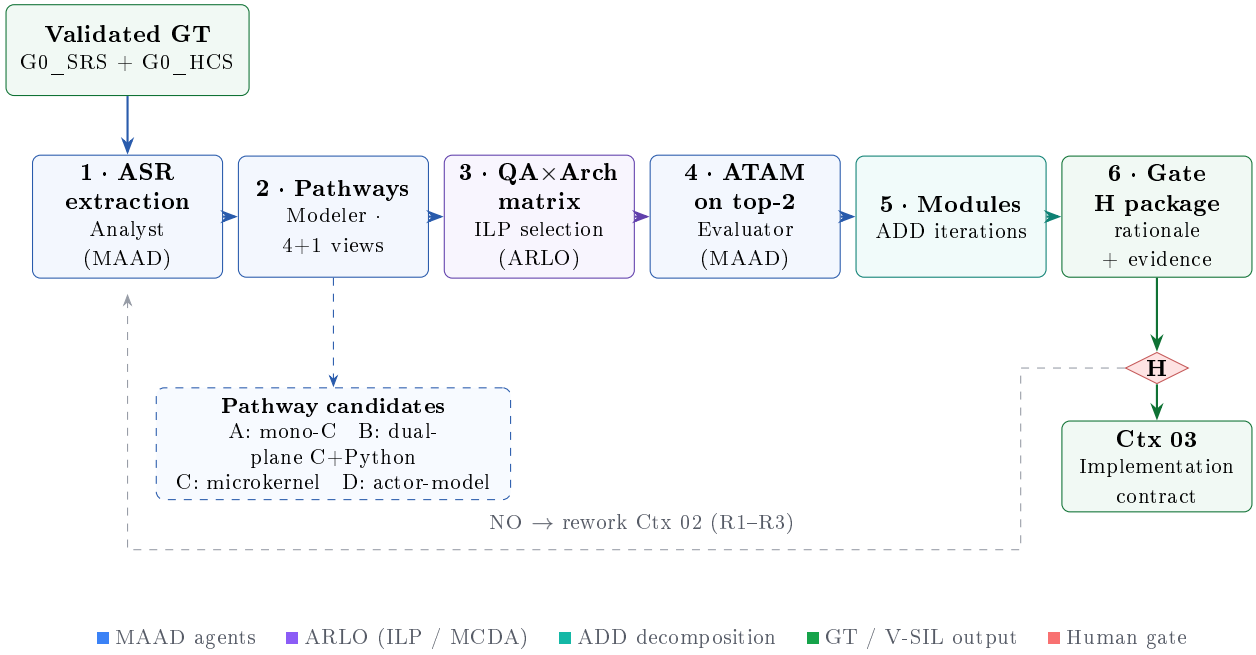


Figure 3: **G1 design pipeline (hybrid MAAD + ARLO + ADD).** The validated GT feeds six steps: extract architecturally significant requirements, generate 3–4 candidate pathways with 4+1 views, score them in a QA×architecture matrix solved by ILP, run ATAM on the top two, decompose the winner into modules, and package everything for human Gate H. A NO routes defects back into the stage (R1–R3).

Table 3: Why G1 hybrids four frameworks: none covers Ctx 02 alone.

Criterion	**MAAD**	**ARLO**	**ADD+LLM**	**SDD**
Multi-hypothesis	Native (mismatch report)	Native (ILP candidates)	Needs adaptation	No
Module decomposition	4+1 views → components	Style only, no modules	Fine-grained per iteration	Contract-level only
Quantitative selection	Qualitative ATAM	ILP optimization (MCDA)	Implicit	N/A
HW-constraint support	Via RAG injection	Via QA mapping	Via driver ID	Via spec contracts
Trade-study output	Sensitivity / risk catalog	Scored matrix + solution	Pattern rationale	Pass/fail per contract
Ctx 02 coverage	Full	Selection only	Decomposition only	Verification only